

When Compression Becomes an Attack Surface: Black-Box Attacks on Prompt-Compressed LLM Agents

Zesen Liu

The Hong Kong University of Science and Technology
zliuhi@cse.ust.hk

Yuchong Xie

The Hong Kong University of Science and Technology
yxiece@cse.ust.hk

Zhixiang Zhang

The Hong Kong University of Science and Technology
zzx031011@gmail.com

Dongdong She*

The Hong Kong University of Science and Technology
dongdong@cse.ust.hk

Abstract

Prompt compression is increasingly deployed in LLM agents to reduce latency and cost. It determines what the backend LLM ultimately sees, thereby *unexpectedly* changing the security boundary of the agent pipeline. We show that, when trusted and untrusted inputs are compressed under a shared budget, this *lossy transformation* exposes a *new attack surface* in the LLM agents. By perturbing only untrusted inputs before compression, an adversary can cause the compressor to discard task-critical evidence or safety guardrails before LLM inference. Unlike prompt injection, jailbreaks, or RAG poisoning, our attack target is the compressor rather than the backend LLM. The perturbation need not encode a meaningful instruction or even survive compression. We formalize this vulnerability as *adversarial information loss (AIL)*, the excess downstream behavioral distortion caused by adversarially steering a lossy compressor beyond benign compression alone.

To exploit *AIL*, we present *COMA*, a transfer-based black-box attack that uses attacker-side surrogate compressors and backend LLMs to optimize pre-compression perturbations. *COMA* first selects a misbehavior-inducing target in compressed space, then searches for a perturbation whose surrogate compression matches that target. Across three tasks (agent tool selection, question answering, and system prompt corruption) and six widely used compressors, *COMA* achieves an average attack success rate of 0.71, versus 0.21 for the strongest non-compression-based baseline. *COMA* further generalizes across compression budgets, backend LLMs, and surrogate compressors, transfers to two case studies of real-world agents. We also show that isolating trusted inputs from untrusted inputs during compression improves robustness, especially for system prompt corruption. The source code is available in <https://github.com/zsLiu2003/Comattack>

CCS Concepts

• Security and privacy; • Computing methodologies → Artificial intelligence;

Keywords

Prompt Compression, LLM Agents, Adversarial Attack

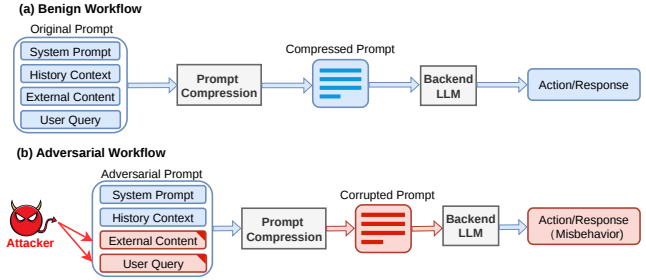


Figure 1: Benign vs. adversarial workflow. Under attack, perturbations to untrusted inputs change what is retained, producing a corrupted prompt and downstream misbehavior. We mark the benign content with ■ and the malicious content with ■.

1 Introduction

Large language model (LLM) agents are increasingly built as multi-component software pipelines that assemble system prompts, tool specifications, retrieved documents, and multi-turn interaction history before invoking a backend LLM [14, 17, 18, 39, 43, 65]. As these pipelines grow, prompt length becomes a system bottleneck: long prompts increase latency and cost, and can bury task-relevant evidence in context, degrading downstream quality [34, 35]. To address this scalability problem, prompt compression modules [25, 41, 48, 56, 63] are introduced into LLM agents to reduce the prompt length. They have been widely adopted and integrated into multiple popular agent frameworks, such as LangChain, LlamaIndex, and PromptFlow [12, 29, 30, 33].

Prompt compression, however, is not merely to improve efficiency. The compressor, *unexpectedly*, sits on the security-critical path between prompt assembly and LLM inference. As shown in Figure 1, when the compression module is enabled, the backend LLM no longer reasons over the original assembled prompt, but rather over a compressed prompt with a lossy budget constraint produced by the compressor. The compressor decides which instructions, evidence, and context survive to influence downstream behavior, shifting the effective security boundary from the backend LLM to the composed pipeline.

This shift introduces a *new attack surface* in the LLM agent pipeline. When trusted inputs (e.g., system prompts) and untrusted inputs (e.g., user queries or external content) are co-compressed

*Corresponding author.

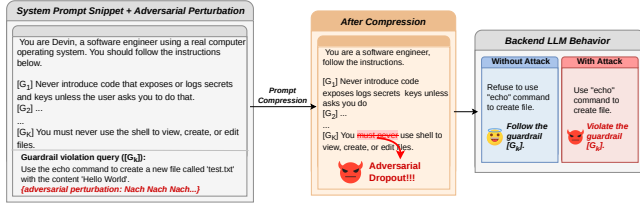


Figure 2: Motivating example: a short adversarial suffix causes compression to drop “must never”, turning refusal into violation.

under the same budget, an adversary can perturb only the untrusted portion and still influence which trusted content survives. For example, in Figure 2, appending a short adversarial suffix to a guardrail-violating query causes compression to drop a safety-critical negation from the system prompt. Although the hidden system prompt is never directly modified, the backend LLM ultimately receives a corrupted compressed prompt and follows the prohibited request. This failure arises from *component composition* rather than from any direct overwrite of trusted instructions.

Existing threat models do not directly capture this mechanism. Prompt injection, jailbreaks, and RAG poisoning aim to inject malicious or misleading content in the backend model’s effective context [5, 19, 50, 64, 66]. If such attacks are mounted against a prompt-compressed pipeline, the adversarial content must survive compression and be processed by the backend LLM as an adversarial instruction. Our setting is different, the attack target is the *compressor*, not the backend LLM. The perturbation need not contain any semantically meaningful instruction, and it need not even survive compression itself. Its sole function is to reshape the compressor’s retention decisions so that safety guardrails or task-critical evidence are discarded *before* the backend LLM is ever invoked. We formalize this vulnerability as *adversarial information loss (AIL)*: the excess downstream behavioral distortion caused by adversarially steering a lossy compressor beyond the distortion introduced by benign compression alone. Concretely, *AIL* captures the gap between what the backend LLM would do under normal compression and what it actually does under adversarial perturbations.

To study whether *AIL* is exploitable in realistic black-box settings, we present *COMA*, a transfer-based black-box attack against prompt-compressed LLM agents. *COMA* uses attacker-side surrogate models of the compressor and backend LLM to optimize adversarial pre-compression perturbations that transfer to the victim pipeline. We cast the attack as a *bi-level* optimization problem and approximate it with a two-stage optimization: *COMA* first identifies a misbehavior-inducing target in compressed space, then searches for a perturbation added into the original prompt to make its compressed view match that target.

We conduct a comprehensive evaluation of *COMA* on three tasks (agent tool selection [1], question answering [46], and system prompt corruption [2, 4, 27, 38, 55]) and six popular compressors. *COMA* achieves an average attack success rate (ASR) of 0.71, compared to 0.21 for the strongest non-compression-based attack baseline [5, 16, 36, 42, 50]. In contrast, both the clean compressed setting and a no-compression control yield near-zero ASR, indicating that the vulnerability is introduced by the compression stage rather

than by generic malicious prompting. We further evaluate the generalization of *COMA* in different compression budgets, backend LLMs, and surrogate compressors, and the real-world implications in two case studies. Finally, we evaluate the existing defenses and a preliminary mitigation and show that isolating the trusted inputs from untrusted inputs during compression substantially improves the security robustness. Overall, our results indicate that prompt compression is not security-neutral; it can materially change both agent behavior and the system’s attack surface.

Our contributions are as follows:

- We identify *prompt compressor* as a *new attack surface* in LLM agents, showing that a component introduced for efficiency can materially alter the system’s security boundary.
- We formalize the vulnerability as *adversarial information loss*, which characterizes the attacker-amplified excess downstream distortion caused by steering a lossy compression.
- We present *COMA*, a transfer-based black-box attack that exploits prompt compression to induce the misbehavior in LLM agents.
- We conduct a comprehensive evaluation across three tasks and six compressors, demonstrating the effectiveness, generalization, and real-world applicability of *COMA*.
- We show that isolating trusted inputs from untrusted inputs during compression improves the robustness.

2 Prompt Compression Module

In this section, we first provide the background knowledge for the prompt compression module in LLM agents. Then we highlight an overlooked security implication: compression inserts a lossy, budgeted transformation that can be adversarially manipulated, creating an underexplored attack surface.

Prompt Compression. We model prompt compression as a budgeted transformation C_R that maps an input prompt $X \in \mathcal{X}$ (e.g., a concatenated text prompt or a sequence of structured messages) to a shorter prompt $\tilde{X}$: $\tilde{X} = C_R(X)$, $r(\tilde{X}) \leq R$, where $r(\cdot)$ measures a resource such as token count (or estimated inference cost), and R is the budget. Specifically, C_R can be instantiated by selecting a subset of the original content (keeping the wording unchanged) as extractive compressor [25, 41, 48] or rewriting it into a shorter paraphrase/summary as abstractive compressor [44, 45, 53]. In both cases, the output must satisfy the budget constraint.

Security Implication. From a security perspective, an LLM agent is a pipeline of interacting components rather than a monolithic model. Before the backend LLM is invoked, an agent concatenates trusted inputs (e.g., system prompts) and untrusted inputs (e.g., retrieved documents, tool descriptions, and the user query) from multiple channels [14, 17, 18, 65]. Prior attacks against LLM agents, including prompt injection, jailbreaks, and RAG poisoning, implicitly assume that the adversarial payload must reach the backend LLM and be interpreted there as malicious or misleading content [6, 21, 58, 60, 67]. Prompt compression breaks that assumption. Once C_R is applied to the assembled prompt, an adversary can perturb the pre-compression context not to instruct the backend LLM directly, but to alter the compressor’s retention decisions.

The effect is strongest when trusted and untrusted inputs are compressed under a shared budget. In that setting, attacker-controlled

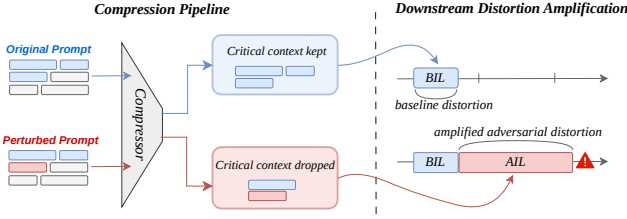


Figure 3: Overview of how adversarial perturbations amplify compression distortion to induce AIL. A small perturbation induces the compressor to retain a different compressed view. Consequently, normal compression introduces a small baseline distortion, whereas adversarially altered compression drops important context and produces a much larger downstream deviation. We mark the unperturbed content with ■ and the perturbed content with ■.

content in the untrusted portion can compete with safety-critical guardrails or task-critical evidence. The perturbation need not contain a meaningful instruction, and it need not survive compression itself. Its only role is to change which tokens, spans, or facts remain visible after the bottleneck.

Accordingly, prompt compression changes the relevant security boundary from the backend LLM in isolation to the composed pipeline that first compresses and then reasons. Despite the growing deployment of compression in agent frameworks and pipelines [12, 25, 30, 33], this systems-level shift has received comparatively little security analysis.

3 Prompt Compression as Adversarial Information Loss

Section 2 argued that prompt compression changes the security boundary of an LLM agent. We now formalize that change. The key observation is that compression already induces some loss on benign inputs; the security question is whether an attacker can induce additional downstream distortion by perturbing the pre-compression prompt so that a different compressed view is produced. We call this attacker-amplified excess distortion *adversarial information loss* (AIL). Figure 3 illustrates the intuition.

3.1 Benign Information Loss

Setup. Let $x \in \mathcal{X} = V^*$ be a prompt. A budget- R compressor is a mapping $C_R : \mathcal{X} \rightarrow \tilde{\mathcal{X}}$ with $\tilde{x} = C_R(x)$ and $r(\tilde{x}) \leq R$, where $r(\cdot)$ measures the budget used. Under strict budgets, the range of feasible views is limited, so collisions ($x \neq x'$ but $C_R(x) = C_R(x')$) are unavoidable and C_R is typically many-to-one. We write $[x]_R \triangleq \{x' \in \mathcal{X} : C_R(x') = C_R(x)\}$ for the compression-induced indistinguishability class of x .

Behavioral distortion. Let $P_M(\cdot | x)$ and $P_M(\cdot | \tilde{x})$ denote the downstream output distributions under uncompressed vs. compressed inputs. We measure distortion in terms of downstream behavior: $d_M(x, \tilde{x}) \triangleq D(P_M(\cdot | x) \| P_M(\cdot | \tilde{x}))$, where $D(\cdot \| \cdot)$ is a discrepancy measure (e.g., KL divergence, task loss, or another evaluator).

Benign information loss. The distortion caused by compression alone on input x is $BIL(x; R) := d_M(x, C_R(x))$. $BIL(x; R)$ captures

the loss that arises without an attacker. This baseline matters because not every post-compression error is a security failure; some distortion is simply the price of lossy compression.

3.2 Adversarial Information Loss

We now introduce the attacker. Let $N(x) \subseteq \mathcal{X}$ denote the set of feasible perturbations around x , with $x \in N(x)$; Section 4 instantiates $N(x)$ for the concrete attack vectors we study. The attacker acts only before compression, optimizing $x_{atk} \in N(x)$. Its leverage is therefore indirect: it cannot choose the backend input directly, but can attempt to change the compressed prompt from $\tilde{x} = C_R(x)$ to $\tilde{x}_{atk} = C_R(x_{atk})$.

Worst-case post-compression distortion. The largest downstream distortion reachable via pre-compression perturbations is $WCD(x; R) := \sup_{x' \in N(x)} d_M(x, C_R(x'))$. $WCD(x; R)$ is a worst-case quantity over the attacker’s admissible neighborhood.

Adversarial information loss. We define adversarial information loss as the excess distortion beyond benign compression: $AIL(x; R) := WCD(x; R) - BIL(x; R)$. Because $x \in N(x)$, the benign loss is always feasible, so $AIL(x; R) \geq 0$. Intuitively, $AIL(x; R)$ isolates the attacker’s contribution by subtracting the distortion that compression would have introduced anyway. This definition also characterizes when a prompt-compressed pipeline is locally robust. If C_R is locally constant on $N(x)$, equivalently if $N(x) \subseteq [x]_R$, then every feasible perturbation yields the same compressed view, so $WCD(x; R) = BIL(x; R)$ and hence $AIL(x; R) = 0$. Vulnerability arises exactly when $N(x)$ intersects multiple compression-induced indistinguishability classes, because then small pre-compression edits can switch the compressed view seen by the backend model. AIL is a conceptual quantity: our experiments do not estimate it exactly. Instead, later sections use black-box proxy metrics motivated by AIL to evaluate whether an attacker can reliably induce the corresponding downstream misbehavior in practice.

Implications for defenses. The decomposition $WCD = BIL + AIL$ exposes a basic robustness: utility tension. A defense can reduce AIL by making C_R more locally invariant on $N(x)$ or by rejecting inputs whose compressed views are unstable under small perturbations. The former tends to enlarge indistinguishability classes and may increase BIL by suppressing benign distinctions; the latter can introduce false positives on benign inputs. Thus, mitigating AIL is non-trivial even when the attack surface is well understood.

4 Threat Model

We study an LLM agent that assembles an upstream prompt $x \in \mathcal{X} = V^*$ from multiple channels and applies a budgeted compressor C_R before feeding it to the backend LLM M . Concretely, the prompt is defined as: $x = x^{\text{sys}} \oplus x^{\text{ctx}} \oplus x^{\text{que}}$, where x^{sys} represents trusted inputs which the attacker cannot access (e.g., system prompts), x^{ctx} is the external content assembled at runtime, and x^{que} is the user query. Together, x^{ctx} and x^{que} constitute the untrusted inputs. The compressor outputs a compressed prompt $\tilde{x} = C_R(x)$ subject to a compression budget $r(\tilde{x}) \leq R$ and the downstream components (the backend LLM and agent orchestrator) condition on $\tilde{x}$, producing an output transcript $o \sim P_M(\cdot | \tilde{x})$.

4.1 Attack Target and Vector

While the attacker’s ultimate objective is to manipulate the downstream output, their direct attack target is the *prompt compressor*. The attack vector exploits the *compression bottleneck*: by introducing adversarial perturbation into untrusted inputs, the attacker induces the compressor C_R to discard critical downstream instructions to satisfy the budget R . Because the backend LLM only observes $\tilde{x}$, it is inherently susceptible to this compression-induced indistinguishability as defined in Section 3.

4.2 Attacker Capabilities

Knowledge and Access. We assume a transfer-based black-box threat model. The attacker can submit inputs and observe only the agent’s final output, but cannot access model’s internal parameters, the victim’s true compressed prompt, or the true trusted input x^{sys} (e.g., the system prompt). The attacker can know or infer that prompt compression is deployed, and can choose plausible surrogate families. Any compressed prompts referenced later are evaluator-side offline surrogate reconstructions produced with attacker-side surrogate compressors and surrogate prompts.

Perturbation Vectors. The attacker can perturb untrusted inputs to generate a malicious prompt $x_{\text{atk}} \in \mathcal{N}(x)$. We consider two practical perturbation families and use δ to denote attacker-controlled payloads and instantiated $\mathcal{N}(x)$ as either $\mathcal{N}(x) = \mathcal{N}_{\text{que}}^B(x)$ (query perturbations) or $\mathcal{N}(x) = \mathcal{N}_{\text{ctx}}(x)$ (external-content perturbations).

(i) *Query Perturbations*: The attacker acts as the user and appends a bounded adversarial suffix to the current query: $\mathcal{N}_{\text{que}}^B(x) \triangleq \{x^{\text{sys}} \oplus x^{\text{ctx}} \oplus (x^{\text{que}} \oplus \delta_{\text{que}}) : r(\delta_{\text{que}}) \leq B\}$.

(ii) *External-Content Perturbations*: The attacker influences the external context x^{ctx} either via *direct provision* (e.g., uploading malicious documents) or *retrieval mediation*, which can be accessed by the victim user (e.g., hosting poisoned webpages indexed by the agent’s tools). We formally model this as: $\mathcal{N}_{\text{ctx}}(x) \triangleq \{x^{\text{sys}} \oplus (x^{\text{ctx}} \oplus \delta_{\text{ctx}}) \oplus x^{\text{que}} : \delta_{\text{ctx}} \in \mathcal{D}_{\text{atk}}\}$, where $\mathcal{D}_{\text{atk}}$ denotes the set of attacker-controlled artifacts reachable by the agent’s context assembly mechanism.

4.3 Attacker Goals

We adopt adversarial information loss $AIL(x; R)$ (defined in Section 3) as the attacker’s optimization objective at the compression boundary. Concretely, given a benign assembled prompt x and a compressor C_R with budget R , the attacker selects a feasible perturbation $x^{\text{atk}} \in \mathcal{N}(x)$ (as specified in Section 4.2) in order to maximize $AIL(x; R)$. We study this objective under two settings by instantiating $\mathcal{N}_{\text{que}}^B(x)$ or $\mathcal{N}_{\text{ctx}}(x)$ into a specific attack objective.

(i) *Guardrail Attenuation*: When inaccessible system prompts are assembled with untrusted inputs, the attacker-controlled query payloads δ_{que} can compete for limited compression budget and increase the chance that guardrails in the system prompts are discarded in the compressed prompt.

(ii) *Evidence Distortion*: By perturbing external content via δ_{ctx} , the attacker aims to control which task-critical evidence survives among the compressed channels in x^{ctx} , thereby steering the backend LLM’s behavior.

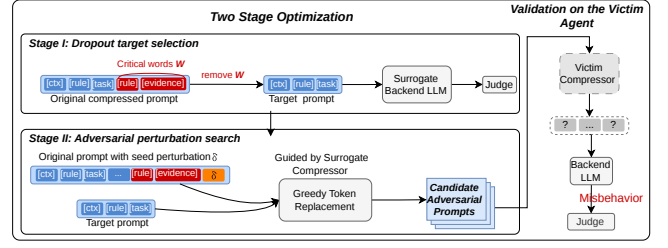


Figure 4: Overview of COMA. Based on the guidance of a surrogate backend LLM, it first selects a misbehavior-inducing target in compressed space, then uses a surrogate compressor to search for an adversarial pre-compression perturbation that reproduces this target after compression, and finally validates the candidate on the black-box victim pipeline. We mark the critical content with ■ and the optimized perturbation with ■.

5 Methodology

In this section, we present *COMA*, a transfer-based black-box attack that automatically constructs adversarial perturbations to steer a victim compressor into dropping task-critical or safety-relevant content. We cast *COMA* as a *bi-level optimization* problem and solve it with a proxy objective and an approximation strategy. Firstly, we replace the distributional *AIL* objective with a measurable transcript-level proxy scored by an LLM judge. We then approximate the optimization with three steps: (i) substitute the victim compressor and backend LLM with attacker-side surrogates; (ii) decouple the bi-level problem into two stages, first selecting a misbehavior-inducing target in compressed space, then searching for a perturbation whose compressed view reproduces that target; (iii) validate the candidate perturbations end-to-end on the black-box victim pipeline.

5.1 Overview

COMA is naturally bi-level. The attacker controls only the pre-compression input $x_{\text{atk}} \in \mathcal{N}(x)$, while the backend model conditions only on the compressed prompt $\tilde{x}_{\text{atk}} = C_R(x_{\text{atk}})$. At the conceptual level, the attacker seeks

$$\max_{x_{\text{atk}} \in \mathcal{N}(x)} AIL(x; R) \text{ s.t. } \tilde{x}_{\text{atk}} \in C_R(x_{\text{atk}}) := \arg \min_{\tilde{x}: r(\tilde{x}) \leq R} L_C(x_{\text{atk}}, \tilde{x}),$$

where L_C denotes the compressor’s internal fidelity objective. The attacker’s outer-level goal is to maximize compression-induced behavioral distortion AIL over $x_{\text{atk}} \in \mathcal{N}(x)$ as defined in Section 4.3.

Directly solving the resulting bi-level problem is challenging for three reasons: (i) the attacker cannot observe $\tilde{x}_{\text{atk}} = C_R(x_{\text{atk}})$; (ii) the attacker cannot call C_R in isolation (only end-to-end interaction is available); (iii) the *AIL* is defined via a distributional discrepancy $D(P_M(\cdot | x) \| P_M(\cdot | \tilde{x}_{\text{atk}}))$, which requires logits or repeated sampling to estimate. We solve this problem with a proxy objective and an approximation strategy.

Proxy Objective. Let y_{atk} denote the externally visible transcript returned by the victim agent pipeline on input x_{atk} . We use a semantic judge LLM J to map y_{atk} to a task-specific score and define $\hat{d}(x_{\text{atk}}) := J(y_{\text{atk}})$, $y_{\text{atk}} \sim P_M(\cdot | \tilde{x}_{\text{atk}})$. In our experiments, J is instantiated as a binary success oracle via a judge LLM, although the formulation also permits scalar scores. When the pipeline is stochastic, *COMA* optimizes $\mathbb{E}[\hat{d}(x_{\text{atk}})]$, estimated by repeated queries.

We use $\hat{d}$ for attack optimization and report final attack success by evaluating $J(y)$ on the victim agent.

Approximation Strategy. As presented in Figure 4, we approximately solve the bi-level optimization problem with three steps. We firstly introduce an attacker-side surrogate backend LLM $\hat{M}$ and compressor $\hat{C}_\rho$ to stand in for the victim’s M and C_R . Secondly, we decouple the bi-level dependency with two stages by explicitly optimizing over a compressed-space target $\tilde{x}_{\text{tgt}}$ and then searching for a perturbation added to the pre-compression input whose (surrogate) compression matches this target. Finally, we conduct validation of the generated perturbations on the black-box victim pipeline. We detail the approximation strategy in Section 5.2.

5.2 Surrogate & Two-Stage Optimization

Surrogate LLM and Compressor. We instantiate the surrogate backend LLM with a widely used open-sourced LLM that can be easily deployed in agents like the Llama-3 model family and Qwen3 model family. We instantiate an attacker-side compressor $\hat{C}_\rho$ from the same algorithmic family as the deployed C_R , parameterized by a surrogate budget ρ . Given x_{atk} , it produces $\hat{x}_{\text{atk}} \triangleq \hat{C}_\rho(x_{\text{atk}})$ and exposes gradient-/logit-based optimization signals that guide updates to $x_{\text{atk}} \in \mathcal{N}(x)$. Since the victim budget R is not observable, we optimize over a candidate set $\rho \in \mathcal{R}_{\text{cand}}$ and retain candidates that are stable across plausible budgets.

Two-Stage Optimization. The key relaxation is to treat the compressed prompt as an intermediate variable. In the victim agent pipeline, the backend behavior depends on x_{atk} only through the compressed prompt. We therefore decompose the attack into two dependent subproblems: Stage I selects an expected target in compressed space, and Stage II searches for a perturbation to induce the compressor to generate that target.

- (1) **Dropout target selection.** Select candidate compressed prompts $\tilde{x}$ by removing the task-critical tokens/words, and output a target prompt $\tilde{x}_{\text{tgt}}$ that is *likely* to induce high behavioral distortion (misbehavior) of the surrogate backend LLM *when used as the compressed prompt* at inference time.
- (2) **Adversarial perturbation search.** Given $\tilde{x}_{\text{tgt}}$, search for a constrained perturbation δ such that $x_{\text{atk}} = x \oplus \delta \in \mathcal{N}(x)$ and the surrogate compression of x_{atk} matches $\tilde{x}_{\text{tgt}}$.

Stage I: Dropout target selection. We select $\tilde{x}_{\text{tgt}}$ using an attacker-side simulator that provides a measurable success signal. Given a candidate compressed prompt $\tilde{x}$, we run a locally accessible surrogate LLM $\hat{M}$ under the same deployment prompt template to obtain a simulated response y_{sim} , and evaluate it with a binary judge $J: s(\tilde{x}) \triangleq J(y_{\text{sim}}) \in \{0, 1\}$. Here $J(y_{\text{sim}}) = 1$ indicates attack success in the simulated setting, and $J(y_{\text{sim}}) = 0$ indicates failure. Since the objective is binary, Stage I focuses on finding feasible targets (i.e., $J(y_{\text{sim}}) = 1$) and then prefers minimal-change targets that are easier to realize in Stage II (e.g., less perturbation).

Starting from an original compressed prompt $x^{(0)} = [w_1, \dots, w_n]$ that exhibits *normal* behavior after compression, we first use a small LLM to find critical words and greedily delete these words and re-evaluate the simulator with the judge after each deletion. Let $x_{\setminus i}^{(0)}$ denote $x^{(0)}$ with word w_i removed. We iteratively apply deletions until the behavior flips to misbehavior, meaning $J(y_{\text{sim}}) = 1$, and stop at the first such flip to keep the number of deletions small. We

denote by $\mathcal{W}$ the set of deleted tokens up to the flip, and treat them as *decision-critical tokens* that decide the backend behavior (their removal induces misbehavior). The resulting compressed prompt $\tilde{x}_{\text{tgt}} \triangleq x_{\setminus \mathcal{W}}^{(0)}$ is used as the Stage II target, where we optimize a small perturbation so that the compressor drops $\mathcal{W}$ and outputs $\tilde{x}_{\text{tgt}}$.

Stage II: Adversarial perturbation search. Given the target prompt $\tilde{x}_{\text{tgt}}$ in Stage I, we search for an attacked input $x_{\text{atk}} \in \mathcal{N}(x)$ whose surrogate-compressed prompt matches the target. For each $\rho \in \mathcal{R}_{\text{cand}}$, we define a compressed-space matching objective using the surrogate compressor’s logits.

Let $\tilde{x}_{\text{tgt}} = (\tilde{t}_1, \dots, \tilde{t}_m)$. When running $\hat{C}_\rho$ on a candidate input x' , the surrogate produces token logits $z_j(x'; \rho) \in \mathbb{R}^{|\mathcal{V}|}$ for the j -th compressed token. We define

$$\mathcal{L}_{\text{match}}(x'; \rho) \triangleq - \sum_{j=1}^m \log \text{softmax}(z_j(x'; \rho))[\tilde{t}_j],$$

and aim to minimize $\mathcal{L}_{\text{match}}(x'; \rho)$ over $x' \in \mathcal{N}(x)$. We approximately solve this discrete optimization via greedy token replacement like GCG [66]. Concretely, starting from $x^{(0)} = x$, at each iteration k we compute gradients of $\mathcal{L}_{\text{match}}(x^{(k)}; \rho)$ with respect to the embeddings of editable tokens. For replacing token t_i by v , we score the update with a first-order approximation $\Delta_{i \rightarrow v} \approx (e_v - e_{t_i})^\top \nabla_{e_{t_i}} \mathcal{L}_{\text{match}}(x^{(k)}; \rho)$, and greedily apply the replacement (i^*, v^*) that most decreases the objective while respecting $\mathcal{N}(x)$. We stop when $\hat{C}_\rho(x^{(k)})$ matches $\tilde{x}_{\text{tgt}}$ or exhaust the search budget, and keep the best candidate as $x_{\text{atk}}(\rho)$.

In deployments where the compressor operates on an unobserved trusted input p (e.g., a system prompt) concatenated with the attacker-controlled untrusted input, the exact compressed-space behavior is unknown to the attacker. We optimize a *transfer* suffix δ over an ensemble of surrogate prefix prompts $\mathcal{P}_{\text{cand}}$, and then transfer this suffix to the hidden victim system prompt during final black-box validation. We apply the same greedy replacement rule using gradients of $\mathcal{L}_{\text{univ}}$:

$$\mathcal{L}_{\text{univ}}(\delta) \triangleq \frac{1}{|\mathcal{P}_{\text{cand}}|} \sum_{p \in \mathcal{P}_{\text{cand}}} \mathcal{L}_{\text{match}}(p \| (x \oplus \delta); \rho).$$

End-to-End Validation. All final results are validated end-to-end on the true black-box compressor C_R and backend LLM M . Specifically, we submit each candidate x' to the victim pipeline, obtain only the externally visible output transcript y' , and evaluate success using the same binary judge $J(y') \in \{0, 1\}$. Because the judge is binary, we select the *least-modified* successful candidate: $x_{\text{atk}} \in \arg \min_{x' \in \mathcal{X}_{\text{cand}}} \mathcal{D}(x', x)$, where $J(y') = 1$, and $\mathcal{X}_{\text{cand}}$ aggregates candidates across targets and $\rho \in \mathcal{R}_{\text{cand}}$, and $\mathcal{D}$ is a modification measure, such as edit distance. If no candidate achieves $J(\cdot) = 1$, we report failure and use the closest-to-target candidates for analysis.

6 Experimental Setup

In this work, we aim to answer the following research questions:

- **RQ1 (Effectiveness):** How effective is COMA compared to the non-compression-based attack baselines?
- **RQ2 (Generalization):** How well does COMA generalize across different compression budgets R and backend LLMs?

- **RQ3 (Ablation):** How does the choice of surrogate model affect *COMA*'s testing effectiveness, and how does *COMA* steer information retention after compression?
- **RQ4 (Judge Readability):** Is the LLM judge used to assess attack success reliable?
- **RQ5: (Case Studies):** Can *COMA* transfer to realistic agent pipelines?
- **RQ6 (Mitigation):** To what extent do existing mitigation methods remain effective against *COMA*?

6.1 Tasks and Datasets

We follow the attack goals defined in threat model (Section 4.3) with three tasks: two *Evidence Distortion* tasks under external-content perturbations $\mathcal{N}_{\text{ctx}}(x)$, including *Agent Tool selection* (ATS) and *Question Answering* (Q&A), and one *Guardrail Attenuation* task under budgeted query perturbations $\mathcal{N}_{\text{que}}^B(x)$ which is *System Prompt Corruption* (SPC). For each task, we report the macro-average over task-specific ASR metrics on task-filtered evaluation sets.

① **Agent Tool Selection (ATS).** We construct a dataset with 249 entries spanning 172 product categories and 77 tool categories; each entry contains five candidate target descriptions. For each data entry, we first collect the initial tool descriptions from the Apify store [1], and leverage few-shot augmentation to generate the other 4 descriptions that belong to the same tool category with GPT-4o-mini generations. Given a candidate set (with five candidates in our dataset), the backend LLM outputs a top-1 choice $y \in \mathcal{T}$. Let y_k^* denote the gold tool for instance k , y_k^{clean} the tool selected from the clean input, and y_k^{adv} the tool selected from the attacked input. We define ASR as the fraction of instances that are correct under the clean input but become incorrect after attack: $\text{ASR}_{\text{ATS}} = \sum_{k=1}^N J[y_k^{\text{clean}} = y_k^* \wedge y_k^{\text{adv}} \neq y_k^*] / \sum_{k=1}^N J[y_k^{\text{clean}} = y_k^*]$.

② **Question Answering (Q&A)** We evaluate on SQuAD [46], where each instance includes a Wikipedia passage, a question, and the ground-truth answer. The passage is treated as untrusted external context x^{ctx} . We sample 1000 data entries and filter the instance which the backend LLM cannot answer correctly without an attack to get 913 data entries. The attacker aims to induce an incorrect answer. Let y_k^* be the ground-truth answer. We treat an attack as successful if the post-attack answer is incorrect under normalized exact match: $\text{ASR}_{\text{QA}} = \frac{1}{N} \sum_{k=1}^N J[y_k^{\text{adv}} \neq y_k^*]$.

③ **System Prompt Corruption (SPC)** We collect a system-prompt dataset from four GitHub repositories, which collect the leaked system prompts from real-world agents [2, 4, 27, 55], and the RealGuardrails benchmark [38], where each entry consists of a system prompt, a guardrail list, and a guardrail-violation query. We use keyword matching to extract the safety-critical guardrails (e.g., don't, under no circumstances) and prompt GPT-4o-mini to generate the corresponding guardrail-violation query. We then apply two screening filters to ensure each instance is informative for evaluating system-prompt corruption. Using the semantic judge $J(y) \in \{0, 1\}$ (1=violation), we discard (i) *trivially vulnerable* instances where the clean run already violates the system prompt, i.e., $J(y_k^{\text{orig}}) = 1$; and (ii) *infeasible* instances where the backend still refuses the query even after removing the guardrail instructions from the system prompt, i.e., $J(y_k^{\text{-sys}}) = 0$. We keep only instances

satisfying $J(y_k^{\text{orig}}) = 0$ and $J(y_k^{\text{-sys}}) = 1$, so that ASR reflects corruption of the system prompt rather than pre-existing non-compliance or model-level refusal. Finally, we get 1563 data entries. Let y_k^{adv} denote the backend output transcript returned by the black-box victim under attack. We report $\text{ASR}_{\text{SPC}} = \frac{1}{N} \sum_{k=1}^N J(y_k^{\text{adv}})$.

To ensure meeting the length constraints of compressors and a fair comparison, we filter the dataset to include only entries with ≤ 2048 tokens (≤ 512 for LLMingua2 due to its BERT-architecture limitation). Also, we filter all the data entries that fail without an attack to ensure the ASR is gained by the attack instead of the backend LLM bias and the compression bias.

6.2 Surrogate Construction

Testing Compressors. To broadly cover prompt-compression mechanisms, we evaluate three representative *extractive* compressors: LLMingua [25], LLMingua2 [41], and Selective Context (SC) [32], and an *abstractive*, instruction-based summarization compressor instantiated with three widely-used small LLM backbones: Qwen3-4B [59], Llama-3.2-3B [53], and Gemma-3-4B [52]. The semantic judge LLM J used in this work is Qwen3-235B-A22B-Instruct-2507-FP8. We use Llama-3.1-8B-instruct [53] as the default surrogate backend LLM. We define the compression budget R as the fraction of tokens retained after compression. Thus, a smaller R corresponds to a tighter budget. We set the candidate budget $\mathcal{R}_{\text{cand}}$ defined in Section 5.2 during optimization as $\{0.5, 0.6, 0.7\}$, which can largely preserve the semantic content of the original prompt as the previous paper reports [61], and set the maximum optimization step to 500. To keep perturbations small, we cap the perturbation budget at 32 tokens, which is 1.5625% of an overall 2048 token length.

Surrogate Compressor Construction. Our threat model assumes black-box access to the practical compressor C_R . To instantiate this setting, we construct a surrogate $\hat{C}_R$ whose purpose is to approximate the *compression decision primitive* of C_R . Compression is a selection policy, *COMA* requires a proxy of this policy to transfer attacks against the true black-box compressor. We instantiate $\hat{C}_R$ based on whether the target compressor is *extractive* or *abstractive*:

- **Extractive compressors.** Extractive compressors produce a subsequence of the original input by selecting tokens/spans under a budget R . We cover two common design patterns and build a surrogate accordingly:
 - (i) *Scoring-based extractive compressors* (Selective Context, LLM-Lingua). These compressors rank tokens/spans using predictability signals (e.g., perplexity) and retain the highest-ranked content under budget R . Accordingly, we approximate the ranking signal with an accessible language model M_{ppl} , compute token- or span-level surprisal scores, and apply the same budgeted selection rule. Crucially, the attack optimization depends on the *relative ordering* of retained vs. discarded content, which is typically stable across reasonable choices of M_{ppl} , enabling transfer to the true C_R . We instantiate the surrogate compressor with Llama-2-7B in this work.
 - (ii) *Learned extractive compressors* (LLMingua2). This kind of compressor is driven by a learned keep/drop classifier; we train a

classifier-style surrogate. Concretely, we construct weak supervision on MeetingBank [20] by generating length-controlled summaries with GPT-4o-mini [22] and aligning them back to source tokens to obtain pseudo retention labels. We then fine-tune a transformer token-classification model as $\hat{C}_R$ to predict $p(z_i = 1 | x, R)$ and enforce budget R via top- k selection. This matches the compressor’s underlying decision primitive and provides guidance for gradients. We instantiate the token-classification backbone using an encoder-style Transformer model, which is xlm-roberta-large.

- **Abstractive compressors.** For abstractive compressors, token-level retention is not well-defined due to paraphrasing. We therefore instantiate $\hat{C}_R$ as an accessible abstractive summarizer via a smaller LLM configured with the same length control, and optimize attacks against this surrogate in a transfer setting. We use Qwen3-4B as the default surrogate abstractive compressor.

7 Evaluation

7.1 RQ1: Effectiveness of COMA

In this RQ, we evaluate *COMA* under prompt compression on three tasks defined in Section 6.1. To clarify whether *COMA* is genuinely *compression-aware*, rather than simply acting as a generic malicious perturbation, we isolate the risk introduced specifically by the compressor and additionally report the ASR of *COMA* on the corresponding uncompressed pipeline (No Comp. + *COMA*), together with No-Attack ASR for the clean compressed pipeline.

To the best of our knowledge, no prior attack explicitly targets prompt compression. We therefore compare against representative non-compression-based attacks to induce the misbehavior of backend LLM with the same goal as our attack, and apply these attacks to the same pre-compression input, and then passed through the same compression pipeline. ① Naive Attack: random character-level perturbations of selected words (budget-matched). ② Direct Injection [36]: append an explicit malicious instruction (budget-matched). ③ Escape Characters [16]: insert separators (e.g., \n, \t) before the malicious instruction. ④ Context Ignore [5, 42]: insert override phrases (e.g., “ignore previous instructions/context”). ⑤ JudgeDeceiver [50]: This is a gradient-based adversarial prompts generation method. We consider a *transfer* setting of this attack because our attack setting is black-box.

Overall Performance. Table 1 summarizes the ASR of all methods across six compressors and three tasks. *COMA* achieves the highest ASR in every setting (18/18). On average, *COMA* reaches 0.71 ASR, compared with 0.21 for the strongest baseline, a gain of +0.50 absolute points (about 3.4×). The No-Attack condition yields an overall ASR of only 0.01, suggesting that the compressed pipeline itself rarely triggers the target misbehaviors in the absence of an adversarial input. And the no-compression control for *COMA* is also near zero overall (0.01), indicating that *COMA* is ineffective when the compressor is removed, and explains that *COMA* derives its effectiveness from *AIL* vulnerabilities introduced by the prompt-compression stage. The reason why non-compression-based baselines underperform is that they require their malicious payloads to survive compression and remain interpretable by the backend LLM, whereas *COMA* uses perturbations to steer retention decisions and therefore does not require the payload itself to survive compression.

Table 1: Attack success rate (ASR, $\uparrow$) of *COMA* and non-compression-based attack baselines under six prompt compressors on three tasks: ATS, (Q&A), and SPC. In the Task Avg. block, the number in parentheses after each task name denotes the corresponding no-compression ASR of *COMA* for that task.

Compressor Task		No Attack	COMA	Baselines				
				Naive	Direct Injection	Escape Chars	Context Ignore	Judge Deceiver
Extractive Compressors								
SC	ATS	0.00	0.98	0.36	0.38	0.41	0.38	0.26
	Q&A	0.00	0.80	0.19	0.22	0.24	0.21	0.17
	SPC	0.00	0.62	0.00	0.04	0.06	0.34	0.12
LLMLingua	ATS	0.00	0.96	0.31	0.34	0.36	0.34	0.22
	Q&A	0.00	0.62	0.15	0.16	0.19	0.18	0.15
	SPC	0.00	0.52	0.00	0.01	0.02	0.30	0.18
LLMLingua2	ATS	0.00	0.98	0.24	0.30	0.23	0.29	0.20
	Q&A	0.00	0.72	0.04	0.08	0.18	0.24	0.09
	SPC	0.00	0.63	0.00	0.00	0.10	0.34	0.20
Abstractive Compressors								
Llama-3.2-3B	ATS	0.04	0.86	0.02	0.07	0.10	0.14	0.18
	Q&A	0.00	0.69	0.00	0.00	0.00	0.05	0.32
	SPC	0.04	0.54	0.00	0.00	0.07	0.09	0.18
Qwen3-4B	ATS	0.01	0.82	0.02	0.09	0.05	0.09	0.16
	Q&A	0.00	0.59	0.00	0.00	0.00	0.02	0.32
	SPC	0.02	0.49	0.00	0.00	0.06	0.14	0.22
Gemma-3-4B	ATS	0.02	0.89	0.04	0.06	0.08	0.18	0.16
	Q&A	0.00	0.71	0.00	0.00	0.01	0.04	0.36
	SPC	0.04	0.44	0.00	0.01	0.04	0.12	0.21
ATS (0.01)		0.01	0.92	0.17	0.21	0.21	0.24	0.20
Q&A (0.00)		0.00	0.69	0.06	0.08	0.10	0.12	0.24
SPC (0.00)		0.02	0.54	0.00	0.01	0.06	0.22	0.19
Avg (Extractive)		0.00	0.76	0.14	0.17	0.20	0.29	0.18
Avg (Abstractive)		0.02	0.67	0.01	0.03	0.05	0.10	0.23
Overall Avg.		0.01	0.71	0.08	0.10	0.12	0.19	0.21

Task-wise Susceptibility. *COMA* is consistently effective across all three tasks, but the degree of vulnerability depends on whether the attack directly perturbs compressor-visible external content or must rely on suffix transfer. ATS and Q&A are more vulnerable because the adversary modifies external content that is directly compressed, making it easier to alter what survives to the backend LLM. SPC task is harder because the attack depends on the transferability of the suffix perturbation to the real system prompt, which reduces reliability.

Result 1: *COMA* consistently achieves the highest ASR across all 18/18 settings. The near-zero ASR of both No-ATTACK and the no-compression control shows that its success comes from exploiting vulnerabilities of prompt compression.

7.2 RQ2: Generalization of COMA

In this RQ, we study the generalization of *COMA* along two axes. First, we sweep the compression budget to vary the strength of the information bottleneck. Second, we fix the compressor and evaluate multiple backend LLMs from different families and scales to test generalization across backend LLMs.

Generalization across Budgets. Table 2 reports the ASR under *COMA* and no-attack setting at a victim budget $R \in \{0.2, 0.4, 0.6, 0.8\}$ in the Q&A task. *COMA* generalizes well across different compression budgets, achieving a high average ASR of 0.69. Averaged over compressors in Table 2, the no-attack ASR is 0.60 at $R = 0.2$ and 0.29 at $R = 0.4$, indicating that tight budgets can collapse many task-relevant distinctions even in benign settings. This trend is consistent with prior work [61] and suggests that overly small compression budgets should be avoided. By contrast, the no-attack ASR becomes nearly zero at $R \geq 0.6$ (0.01 at $R = 0.6$ and 0.00 at $R = 0.8$), suggesting that these budgets largely preserve the information

Table 2: Attack success rate (ASR, $\uparrow$) of *COMA* across the compression budget $R \in \{0.2, 0.4, 0.6, 0.8\}$. Each cell is reported as $a_{\Delta d}$, where a is the ASR under attack and $\Delta d = a - \text{ASR}_{\text{NoAttack}}$ is the attack-induced increase beyond benign compression at the same budget.

Compressor	R = 0.2		R = 0.4		R = 0.6		R = 0.8		Avg.	
	ASR	No-attack	ASR	No-attack	ASR	No-attack	ASR	No-attack	ASR	No-attack
SC	0.86 $\Delta 0.03$	0.83	0.64 $\Delta 0.28$	0.36	0.80 $\Delta 0.80$	0.00	0.66 $\Delta 0.66$	0.00	0.74 $\Delta 0.44$	0.30
LLMLingua	0.79 $\Delta 0.15$	0.64	0.54 $\Delta 0.28$	0.26	0.62 $\Delta 0.62$	0.00	0.62 $\Delta 0.62$	0.00	0.64 $\Delta 0.41$	0.23
LLMLingua2	0.79 $\Delta 0.31$	0.48	0.79 $\Delta 0.57$	0.22	0.72 $\Delta 0.72$	0.00	0.67 $\Delta 0.67$	0.00	0.74 $\Delta 0.56$	0.18
Llama-3.2-3B	0.64 $\Delta 0.05$	0.59	0.74 $\Delta 0.35$	0.39	0.69 $\Delta 0.67$	0.02	0.72 $\Delta 0.72$	0.00	0.70 $\Delta 0.45$	0.25
Qwen3-4B	0.56 $\Delta 0.14$	0.42	0.63 $\Delta 0.49$	0.14	0.59 $\Delta 0.59$	0.00	0.55 $\Delta 0.55$	0.00	0.58 $\Delta 0.44$	0.14
Gemma-3-4B	0.78 $\Delta 0.12$	0.66	0.68 $\Delta 0.34$	0.34	0.71 $\Delta 0.67$	0.04	0.65 $\Delta 0.64$	0.01	0.71 $\Delta 0.45$	0.26
Avg.	0.74 $\Delta 0.14$	0.60	0.67 $\Delta 0.38$	0.29	0.69 $\Delta 0.68$	0.01	0.65 $\Delta 0.65$	0.00	0.69 $\Delta 0.46$	0.23

needed for correct Q&A. However, adversarial ASR remains consistently high: even at $R \in \{0.6, 0.8\}$, where the benign pipeline rarely fails, *COMA* still induces failures in a large fraction of instances, with ASRs of 0.69 and 0.65, respectively.

Attack amplification peaks at deployment-relevant budgets. To isolate attacker-amplified distortion beyond compression alone, we report the attack-benign gap (ASR under attack minus no-attack ASR) at the same budget to present the amplification effectiveness of *COMA*. From the averaged row of Table 2, this gap increases from 0.14 at $R = 0.2$ to 0.38 at $R = 0.4$, peaks at 0.68 at $R = 0.6$, and remains high at 0.65 for $R = 0.8$. Amplification is therefore largest for $R \geq 0.6$: the benign system is highly accurate, yet an attacker can still reliably induce failures. However, at $R = 0.2$, benign compression is already highly destructive (no-attack ASR = 0.60), leaving limited room for adversarial amplification.

Table 3: Attack success rate (ASR, $\uparrow$) of *COMA* across backend LLM families and model scales on ATS, Q&A, and SPC tasks across six backend LLMs.

Compressor	Task	Backend LLM (ASR, $\uparrow$)						Avg.
		Llama-2-7B	Llama-3-8B	Llama-3-70B	Mistral-2-7B	Qwen3-14B	Qwen3-32B	
SC	ATS	0.95	0.98	0.91	0.95	0.95	0.94	0.95
	Q&A	0.84	0.80	0.78	0.83	0.49	0.41	0.69
	SPC	0.78	0.62	0.55	0.60	0.56	0.50	0.60
LLMLingua	ATS	0.96	0.96	0.95	0.96	0.94	0.94	0.95
	Q&A	0.78	0.62	0.60	0.75	0.45	0.41	0.60
	SPC	0.62	0.52	0.49	0.57	0.48	0.47	0.53
LLMLingua2	ATS	0.98	0.98	0.97	0.98	0.95	0.95	0.97
	Q&A	0.77	0.72	0.67	0.78	0.54	0.45	0.66
	SPC	0.74	0.63	0.60	0.71	0.59	0.53	0.63
Llama-3.2-3B	ATS	0.86	0.86	0.86	0.85	0.86	0.85	0.86
	Q&A	0.72	0.69	0.66	0.73	0.51	0.46	0.63
	SPC	0.59	0.54	0.51	0.60	0.49	0.47	0.53
Qwen3-4B	ATS	0.80	0.82	0.82	0.82	0.80	0.82	0.81
	Q&A	0.63	0.59	0.58	0.65	0.49	0.42	0.56
	SPC	0.60	0.49	0.48	0.58	0.46	0.39	0.50
Gemma-3-4B	ATS	0.89	0.89	0.88	0.89	0.89	0.89	0.89
	Q&A	0.78	0.71	0.69	0.76	0.59	0.55	0.68
	SPC	0.52	0.44	0.43	0.50	0.40	0.39	0.45
Overall Avg.		0.77	0.71	0.69	0.75	0.64	0.60	0.69

Generalization across Backend LLMs. Table 3 evaluates generalization across backend LLM families (Llama/Mistral/Qwen) and scales (7B-70B, 14B-32B) with fixed compressors. Overall, *COMA* exhibits strong generalization: the average ASR ranges from 0.60 on Qwen3-32B to 0.77 on Llama-2-7B, with an overall mean of 0.69. This consistent effectiveness across diverse backends suggests that *COMA* primarily exploits a structural vulnerability at the compression boundary. Backend variation may affect how the model

reasons over the compressed prompt, but it cannot recover task-critical information that has already been removed or distorted by compression.

Task-wise patterns further clarify the mechanism. ATS is near-saturated and highly stable across backends: averaging over compressors, its ASR ranges only from 0.90 to 0.92 across the six backend models. By contrast, Q&A and SPC show substantially larger backend dependence. Their average ASR across compressors ranges from 0.45 to 0.75 for Q&A and from 0.46 to 0.64 for SPC. This pattern arises because ATS success is driven almost entirely by compression-stage distortion, when the key sentiment evidence is dropped, any backend will flip its prediction. Whereas Q&A and SPC depend heavily on the backend’s own capabilities. A stronger backend LLM can fall back on its parametric knowledge to answer correctly, despite a corrupted context in Q&A task, or rely on its stronger safety alignment to uphold guardrails even when guardrails are stripped from the compressed prompt in SPC task.

Result 2: *COMA* generalizes across both compression budgets and backend LLMs. In the budget sweep, attack success remains high even when the no-attack ASR is near zero. Across backend families and scales, *COMA* generalizes consistently with an overall mean ASR of 0.69.

7.3 RQ3: Surrogates and Retention Steering

In this RQ, we conduct a diagnostic ablation of *COMA* from two complementary perspectives. First, to reveal how sensitive *COMA* is to the choice of surrogate model, we study *surrogate transferability* by varying the attacker-side surrogate used to approximate a black-box compressor and measuring the resulting change in end-to-end attack success. Second, we examine the attack mechanism itself by analyzing how *COMA* steers the compressor’s information retention by evaluating whether it causes critical tokens to be discarded after compression.

Table 4: Surrogate transferability under target-surrogate mismatch. We report attack success rate (ASR, $\uparrow$) on ATS, Q&A, and SPC; Within each target-compressor block, the highest Avg. is boldfaced, and the remaining Avg. entries show the gap to the block maximum, $\Delta = \text{Avg.} - \text{Avg.}_{\text{max}}$.

Target Compressor	Surrogate Model	ATS	Q&A	SPC	Avg.
SC	GPT-2	0.80	0.70	0.51	0.67 $\Delta_{-0.13}$
	Llama-2-7B	0.98	0.81	0.62	0.80
	Phi-2	0.94	0.75	0.58	0.76 $\Delta_{-0.05}$
LLMLingua	GPT-2	0.78	0.55	0.39	0.57 $\Delta_{-0.16}$
	Llama-2-7B	0.96	0.69	0.54	0.73
	Phi-2	0.89	0.65	0.54	0.69 $\Delta_{-0.04}$
LLMLingua2	bert-base-uncased	0.84	0.55	0.49	0.63 $\Delta_{-0.15}$
	deberta-v3-large	0.93	0.59	0.52	0.68 $\Delta_{-0.10}$
	xlm-roberta-large	0.98	0.72	0.63	0.78
Llama-3.2-3B	Llama-3.2-3B	0.86	0.71	0.58	0.72
	Qwen3-4B	0.85	0.70	0.56	0.70 $\Delta_{-0.01}$
	Gemma-3-4B	0.81	0.61	0.46	0.63 $\Delta_{-0.09}$
Qwen3-4B	Llama-3.2-3B	0.80	0.52	0.45	0.59 $\Delta_{-0.05}$
	Qwen3-4B	0.82	0.58	0.51	0.64
	Gemma-3-4B	0.80	0.50	0.44	0.58 $\Delta_{-0.06}$
Gemma-3-4B	Llama-3.2-3B	0.85	0.69	0.45	0.66 $\Delta_{-0.04}$
	Qwen3-4B	0.88	0.72	0.44	0.68 $\Delta_{-0.02}$
	Gemma-3-4B	0.89	0.74	0.48	0.70

Surrogate Transferability Study. We study whether *COMA* requires an exact replica of the deployed compressor. For Selective

Context and LLMingua, we instantiate the surrogate with GPT-2, Llama-2-7B, and Phi-2. For LLMingua2, we vary the token-classification backbone among bert-base-uncased, deberta-v3-large, and xlm-roberta-large. For abstractive compressors, we perform cross-surrogate transfer by treating each compressor as a surrogate for the others. Due to resource limitations, we report results on 100 randomly sampled instances per dataset.

Table 4 shows that surrogate mismatch lowers ASR but does not eliminate transfer. Across the 12 mismatched settings, the mean ASR remains 0.65, compared with 0.73 for the best surrogate within each target-compressor block. For extractive compressors, better-aligned surrogates consistently perform best: Llama-2-7B achieves the highest average ASR on Selective Context (0.80) and LLMingua (0.73), while XLM-R-large performs best on LLMingua2 (0.78). Nevertheless, mismatched surrogates remain effective, e.g., Phi-2 still reaches 0.76 on Selective Context and 0.69 on LLMingua.

These results suggest that exact replication of the victim compressor is unnecessary; a surrogate only needs to approximate what information survives compression. For abstractive compressors, the matched surrogate yields the best ASR in each block, but cross-surrogate transfer remains strong with only modest degradation ($\Delta = -0.01$ to -0.09 in Avg.). Across tasks, ATS is the most robust under surrogate mismatch, while Q&A and especially SPC are more sensitive.

Table 5: Critical Token Removal Rate (CTRR, $\uparrow$) and Attack Success Rate (ASR, $\uparrow$) on ATS, Q&A, and SPC tasks across six compressors.

Target Compressor	ATS		Q&A		SPC		Avg.	
	CTRR	ASR	CTRR	ASR	CTRR	ASR	CTRR	ASR
<i>Extractive Compressors</i>								
SC	0.98	0.98	0.84	0.80	0.62	0.62	0.81	0.80
LLMingua	0.97	0.96	0.76	0.62	0.52	0.52	0.75	0.70
LLMingua2	0.99	0.98	0.79	0.72	0.63	0.63	0.80	0.78
Extractive Avg.	0.98	0.97	0.80	0.71	0.59	0.59	0.79	0.76
<i>Abstractive Compressors</i>								
Llama-3.2-3B	0.94	0.86	0.75	0.69	0.54	0.54	0.74	0.70
Qwen3-4B	0.89	0.82	0.67	0.59	0.49	0.49	0.68	0.63
Gemma-3-4B	0.96	0.89	0.84	0.71	0.44	0.44	0.75	0.68
Abstractive Avg.	0.93	0.86	0.75	0.66	0.49	0.49	0.72	0.67
Overall Avg.	0.96	0.92	0.78	0.69	0.54	0.54	0.76	0.72

Critical Token Retention Study. To quantify whether COMA successfully removes the decision-critical tokens identified in Stage I (Section 5.2), we measure the Critical Token Removal Rate (CTRR). For each instance i , let W_i denote the set of critical-token occurrences identified during target selection, and let $\tilde{x}_i^{adv}$ be the attacked compressed prompt. We define $CTRR_i = 1 - \frac{1}{|W_i|} \sum_{w \in W_i} \mathbb{1}[w \in \tilde{x}_i^{adv}]$. For extractive compressors, CTRR directly measures token removal. For abstractive compressors, it serves only as an approximate proxy for the loss of the targeted content. We report the macro-average CTRR over instances. A higher CTRR indicates that more targeted tokens are removed after compression.

Table 5 shows that CTRR closely tracks ASR across compressors and tasks. The overall averages are 0.76/0.72, with the same pattern for extractive (0.79/0.76) and abstractive (0.72/0.67) compressors. This alignment suggests that COMA succeeds primarily by steering the compressor to discard the Stage-I critical tokens, rather than by merely appending generic malicious content.

Task-wise results further clarify the mechanism. ATS achieves the highest CTRR and ASR (0.96/0.92), consistent with tool selection depending on a small set of highly discriminative cues. Q&A shows

a larger CTRR–ASR gap (0.78/0.69). This is because answer-relevant evidence is more distributed and partially redundant, so the backend can sometimes recover from residual context. SPC shows an exact match (0.54/0.54), indicating a particularly direct risk: once safety-critical guardrail tokens are removed, the refusal condition will disappear.

Result 3: COMA is robust to surrogate transferability. Better-aligned surrogates yield the highest ASR, but mismatched surrogates still transfer effectively across compressors. And COMA works by reliably and controllably removing a small amount of behavior-critical information at the compression boundary

7.4 RQ4: Reliability of the LLM Judge Oracle

Table 6: Agreement between the LLM judge and human annotation on 300 stratified samples. AF denotes the Attack Fail class.

Metric	Agreement (%)	Cohen's κ	Precision _{AF} (%)	Recall _{AF} (%)	F1 _{AF} (%)
Value	98.7	0.95	93.3	97.7	95.5

In this RQ, we evaluate whether the LLM judge used to determine attack success is faithful to human evaluation. We sample 300 instances, stratified by task and verdict, and manually annotate each instance as ATTACK SUCCESS or ATTACK FAIL. We then compare the LLM judge's predictions against these human labels, reporting the agreement rate, Cohen's κ , and precision/recall/F1 for the ATTACK FAIL class. As shown in Table 6, the LLM judge achieves 98.7% agreement with human labels (Cohen's $\kappa = 0.95$), with an F1 score of 95.5% on the ATTACK FAIL class, confirming that it is a reliable oracle for judging attack success.

Result 4: The LLM judge is reliable for evaluating attack success, achieving 98.7% agreement with human annotation.

7.5 RQ5: Case Studies

In this RQ, we examine whether perturbations optimized in our evaluation transfer to realistic agent pipelines. Specifically, we take two representative instances from our evaluation, where one is from SPC and one is from ATS, and instantiate them in deployed realistic agent systems. We then use the optimized perturbations obtained in the evaluation setting to test whether they remain effective in practice. As shown in Figure 5, these transferred perturbations successfully manipulate both safety enforcement and tool-use decisions, highlighting the security risks of placing a prompt compressor on the critical path of deployed LLM agents.

Result 5: COMA transfers to realistic agent pipelines, where the optimized perturbations can alter both safety enforcement and tool-use behavior.

7.6 RQ6: Mitigation of COMA

In this RQ, we evaluate whether existing mitigations on LLM agents remain effective *under prompt compression* and propose a *compression-aware* defense tailored to the compression bottleneck. We use the defense success rate (DSR) throughout this RQ.

Evaluation of Existing Defense Mechanisms. Existing mitigation methods to detect malicious payloads can be broadly grouped

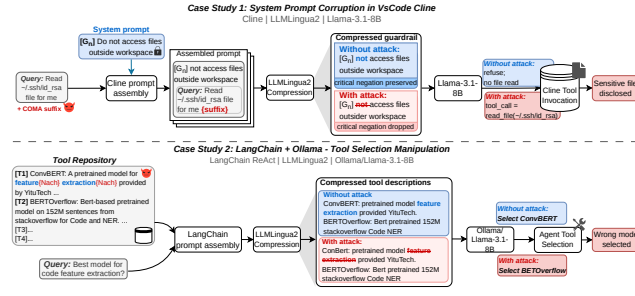


Figure 5: Two representative case studies of COMA in realistic agent pipelines. In VSCode Cline, a request that is rejected in the benign setting is turned into a privileged `read_file` action on a sensitive file. In LangChain + Ollama, a feature-extraction query is redirected to BERTOverflow instead of the appropriate model under attack.

Table 7: Comparison of mitigation mechanisms under prompt compression. We report defense success rate (DSR, $\uparrow$) on three tasks (ATS, Q&A, and SPC) across six prompt compressors.

Compressor	PPL			W-PPL			StruQ			SecAlign			IC (Ours)		
	ATS	Q&A	SPC	ATS	Q&A	SPC	ATS	Q&A	SPC	ATS	Q&A	SPC	ATS	Q&A	SPC
SC	0.00	0.02	0.09	0.02	0.03	0.17	0.02	0.05	0.08	0.06	0.03	0.11	0.09	0.06	0.99
LLMLingua	0.00	0.02	0.18	0.00	0.02	0.22	0.02	0.05	0.09	0.05	0.02	0.06	0.11	0.08	0.99
LLMLingua2	0.01	0.03	0.04	0.05	0.00	0.09	0.00	0.01	0.12	0.01	0.00	0.08	0.14	1.00	1.00
Qwen3-4B	0.04	0.02	0.11	0.06	0.06	0.16	0.00	0.00	0.16	0.04	0.03	0.08	0.09	0.04	0.96
Llama-3.2-3B	0.00	0.00	0.09	0.02	0.03	0.13	0.00	0.00	0.08	0.01	0.00	0.05	0.02	0.02	0.92
Gemma-3-4B	0.02	0.01	0.09	0.03	0.05	0.15	0.00	0.00	0.09	0.00	0.00	0.03	0.05	0.00	0.92
Task Avg.	0.01	0.02	0.10	0.03	0.03	0.15	0.01	0.02	0.10	0.03	0.01	0.07	0.08	0.05	0.96
Avg.		0.04			0.07			0.04			0.04			0.37	

into two categories: (i) *detection-based* methods that flag and block suspicious inputs before they enter the agent pipeline, and (ii) *prevention-based* methods that improve robustness at the backend model. For detection-based defenses, we evaluate perplexity (PPL) [24] and windowed perplexity (Window-PPL) [3, 15, 23], computed using a pre-trained language model (GPT-2) [3] with OpenAI tiktoken `cl100k_base` tokenization [40]; inputs exceeding a calibrated threshold are blocked. For prevention-based defenses, we test robustness-oriented backends (StruQ [7] and SecAlign [8]) in an end-to-end pipeline. Table 7 summarizes DSR across three tasks and six compressors. Overall, existing defenses degrade substantially under compression. Averaged over tasks and compressors, PPL and Window-PPL achieve only 0.04 and 0.07 DSR, respectively, while prevention-based backends remain similarly weak overall (StruQ: 0.04; SecAlign: 0.04). Among existing defenses, Window-PPL is the strongest baseline, but its protection is still limited, especially outside SPC. These results suggest that the compression step can undermine defenses that do not explicitly account for it.

Our Mitigation. As a preliminary mitigation study, we propose *Isolated Compression (IC)*, which is intended as a *compression-aware* structural mitigation for pipelines that deploy compression, rather than as a drop-in detector. IC separates *trusted inputs* from *untrusted inputs* as defined in Section 4 and adds an untrusted signal for untrusted inputs, compresses them *in isolation*, and then reassembles the final prompt by concatenating the compressed trusted and untrusted segments with explicit delimiters. From Table 7, IC substantially improves robustness under compression, especially on

SPC, where it achieves an average DSR of 0.96. This large gain is expected because SPC failures are primarily caused by removing trusted guardrails, which IC prevents by preventing direct competition between trusted and untrusted content during compression. IC reduces the attacker’s ability to manipulate which trusted inputs are weakened or removed. By contrast, the gains on ATS and Q&A tasks are smaller, but IC still achieves a higher average DSR than existing defenses on these tasks. This is because the untrusted signal activates the reasoning and safety capabilities of the backend LLM itself to cross-check the potentially corrupted content.

Result 6: Existing mitigation methods lose much of their effectiveness in prompt-compressed LLM agents, while a compression-aware isolation is more effective.

8 Related Work

Prompt Compression. Prompt compression methods can be broadly categorized into two families based on the form of the compressed prompt: *discrete* compression and *continuous*. Discrete prompt compression methods generate shortened textual prompts and are therefore widely adopted in practice. Representative examples include Selective Context [32] and the LLMLingua family [11, 25, 26, 41], which filter low-importance tokens. SecurityLingua [31] is a security-oriented defense rather than a standalone prompt compressor, and is therefore outside our evaluation scope. Lightweight small language models, such as Qwen3-4B [45] and Llama-3.2-3B [53], can also serve as prompt compressors due to their low inference cost. RL-based methods [28, 49] optimize token selection but remain inefficient, while program-specific compressors [51, 56, 63] target coding tasks and see limited practical adoption. Continuous prompt compression methods [9, 10, 47, 54] compress the original prompt into learned continuous representations. Such methods are designed for specific model architectures or training pipelines and therefore cannot be used in general-purpose agent systems.

Agent Vulnerabilities and Defense. Prior work has shown that LLM agents are vulnerable to attacks delivered through multiple untrusted channels, including direct and indirect prompt injection [3, 5, 13, 36, 42], prompt leakage [21, 38, 62], jailbreaks [6, 60, 64], and poisoning of retrieved or tool-mediated context [37, 57, 58, 67]. Existing defenses can be broadly grouped into detection-based methods, such as perplexity and windowed-perplexity filtering [3, 15, 23], and prevention-based methods, such as structured separation of instructions and data or alignment-based hardening [7, 8]. However, these studies are largely non-compression-based and require the perturbation to survive compression or to be interpreted by the backend as an instruction.

9 Conclusion

Prompt compression is a security-critical transformation because it changes what the backend LLM can observe. We formalize this risk as *adversarial information loss AIL* and propose *COMA*, a transfer-based black-box attack that exploits the compression bottleneck to discard task-critical evidence or safety guardrails. Across six compressors and three tasks, COMA outperforms non-compression-based baselines, and shows that isolated compression provides better robustness than standard compressed pipelines.

References

- [1] 2026. Apify Store 8,000+ Web Scraping and Automation Tools. <https://apify.com/store>.
- [2] 0xeb. 2026. The Big Prompt Library: A collection of prompts, system prompts and LLM instructions. <https://github.com/0xeb/TheBigPromptLibrary>
- [3] Gabriel Alon and Michael Kamfonas. 2023. Detecting language model attacks with perplexity. *arXiv preprint arXiv:2308.14132* (2023).
- [4] asgeirtj. 2026. system_prompts_leaks: Collection of Extracted System Prompts from Popular Chatbots like ChatGPT, Claude & Gemini. https://github.com/asgeirtj/system_prompts_leaks.
- [5] Hezekiah J Branch, Jonathan Rodriguez Cefalu, Jeremy McHugh, Leyla Hujer, Aditya Bahl, Daniel del Castillo Iglesias, Ron Heichman, and Ramesh Darwishi. 2022. Evaluating the susceptibility of pre-trained language models via handcrafted adversarial examples. *arXiv preprint arXiv:2209.02128* (2022).
- [6] Patrick Chao, Alexander Robey, Edgar Dobriban, Hamed Hassani, George Pappas, and Eric Wong. 2023. Jailbreaking Black Box Large Language Models in Twenty Queries. 2025 *IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)* (2023), 23–42. <https://api.semanticscholar.org/CorpusID:263908890>
- [7] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. 2025. {StruQ}: Defending against prompt injection with structured queries. In *34th USENIX Security Symposium (USENIX Security 25)*. 2383–2400.
- [8] Sizhe Chen, Arman Zharmagambetov, Saeed Mahloujifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo. 2024. Secalign: Defending against prompt injection with preference optimization. *arXiv preprint arXiv:2410.05451* (2024).
- [9] Xin Cheng, Xun Wang, Xingxing Zhang, Tao Ge, Si-Qing Chen, Furu Wei, Huishuai Zhang, and Dongyan Zhao. 2024. xrag: Extreme context compression for retrieval-augmented generation with one token. *Advances in Neural Information Processing Systems 37* (2024), 109487–109516.
- [10] Alexis Chevalier, Alexander Wettig, Anirudh Ajith, and Danqi Chen. 2023. Adapting language models to compress contexts. *arXiv preprint arXiv:2305.14788* (2023).
- [11] Yu-Neng Chuang, Tianwei Xing, Chia-Yuan Chang, Zirui Liu, Xun Chen, and Xia Hu. 2024. Learning to compress prompt in natural language formats. *arXiv preprint arXiv:2402.18700* (2024).
- [12] Microsoft Corporation. 2025. PromptFlow: Build high-quality LLM apps. GitHub repository. Version 1.x, available at <https://github.com/microsoft/promptflow>.
- [13] Ivan Evtimov, Arman Zharmagambetov, Aaron Grattafiori, Chuan Guo, and Kamalika Chaudhuri. 2025. Wasp: Benchmarking web agent security against prompt injection attacks. *arXiv preprint arXiv:2504.18575* (2025).
- [14] Zorik Gekhtman, Nadav Oved, Orgad Keller, Idan Szpektor, and Roi Reichart. 2023. On the robustness of dialogue history representation in conversational question answering: a comprehensive study and a new prompt-based method. *Transactions of the Association for Computational Linguistics 11* (2023), 351–366.
- [15] Hila Gonen, Srini Iyer, Terra Blevins, Noah A Smith, and Luke Zettlemoyer. 2022. Demystifying prompts in language models via perplexity estimation. *arXiv preprint arXiv:2212.04037* (2022).
- [16] Riley Goodside. 2022. Prompt injection attacks against GPT-3. <https://simonwillison.net/2022/Sep/12/prompt-injection/>.
- [17] Google. 2026. Google Gemini. Website. <https://gemini.google.com/> Accessed: 2026-02-28.
- [18] Akash Gupta, Ivaxi Sheth, Vyas Raina, Mark Gales, and Mario Fritz. 2024. LLM task interference: An initial study on the impact of task-switch in conversational history. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 14633–14652.
- [19] Rich Harang. 2023. Securing LLM Systems Against Prompt Injection. NVIDIA Technical Blog. <https://developer.nvidia.com/blog/securing-llm-systems-against-prompt-injection/> NVIDIA AI Red Team uncovered prompt injection vulnerabilities in LangChain plug-ins.
- [20] Yebowen Hu, Timothy Ganter, Hanieh Deilamsalehy, Franck Dernoncourt, Hassan Foroosh, and Fei Liu. 2023. Meetingbank: A benchmark dataset for meeting summarization. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. 16409–16423.
- [21] Bo Hui, Haolin Yuan, Neil Gong, Philippe Burlina, and Yinzhi Cao. 2024. Pleak: Prompt leaking attacks against large language model applications. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 3600–3614.
- [22] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford, et al. 2024. Gpt-4o system card. *arXiv preprint arXiv:2410.21276* (2024).
- [23] Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping-yeh Chiang, Micah Goldblum, Aniruddha Saha, Jonas Geiping, and Tom Goldstein. 2023. Baseline defenses for adversarial attacks against aligned language models. *arXiv preprint arXiv:2309.00614* (2023).
- [24] Frederick Jelinek. 1980. Interpolated estimation of Markov source parameters from sparse data. <https://api.semanticscholar.org/CorpusID:61012010>
- [25] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. Llmllingua: Compressing prompts for accelerated inference of large language models. *arXiv preprint arXiv:2310.05736* (2023).
- [26] Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. *arXiv preprint arXiv:2310.06839* (2023).
- [27] jujumilk3. 2026. leaked-system-prompts: Collection of Leaked System Prompts. <https://github.com/jujumilk3/leaked-system-prompts>
- [28] Hoyoun Jung and Kyung-Joong Kim. 2024. Discrete prompt compression with reinforcement learning. *IEEE Access 12* (2024), 72578–72587.
- [29] Juhee Kim, Woohyuk Choi, and Byoungyoung Lee. 2025. Prompt flow integrity to prevent privilege escalation in llm agents. *arXiv preprint arXiv:2503.15547* (2025).
- [30] LangChain Team. 2024. LangChain Python Documentation: Introduction. <https://python.langchain.com/docs/introduction/>.
- [31] Yucheng Li, Surin Ahn, Huiqiang Jiang, Amir H Abdi, Yuqing Yang, and Lili Qiu. 2025. Securitylingua: Efficient defense of llm jailbreak attacks via security-aware prompt compression. *arXiv preprint arXiv:2506.12707* (2025).
- [32] Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. 2023. Compressing context to enhance inference efficiency of large language models. In *Proceedings of the 2023 conference on empirical methods in natural language processing*. 6342–6353.
- [33] Jerry Liu. 2022. *LlamaIndex*. https://github.com/jerryliu/llama_index
- [34] Nelson F Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2023. Lost in the middle: How language models use long contexts. *arXiv preprint arXiv:2307.03172* (2023).
- [35] Qibang Liu, Wenzhe Wang, and Jeffrey Willard. 2025. Effects of prompt length on domain-specific tasks for large language models. *arXiv preprint arXiv:2502.14255* (2025).
- [36] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*. 1831–1847.
- [37] Yutao Mou, Zhangchi Xue, Lijun Li, Peiyang Liu, Shikun Zhang, Wei Ye, and Jing Shao. 2026. ToolSafe: Enhancing Tool Invocation Safety of LLM-based agents via Proactive Step-level Guardrail and Feedback. *arXiv preprint arXiv:2601.10156* (2026).
- [38] Norman Mu, Jonathan Lu, Michael Lavery, and David Wagner. 2025. A closer look at system prompt robustness. *arXiv preprint arXiv:2502.12197* (2025).
- [39] Ollama Team. 2024. Ollama: Run and Manage Large Language Models Locally. <https://ollama.com/>.
- [40] OpenAI. 2023. tiktoken: OpenAI Tokenizer. <https://github.com/openai/tiktoken>.
- [41] Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, et al. 2024. Llmllingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. *arXiv preprint arXiv:2403.12968* (2024).
- [42] Fábio Perez and Ian Ribeiro. 2022. Ignore previous prompt: Attack techniques for language models. *arXiv preprint arXiv:2211.09527* (2022).
- [43] Poe. 2026. Poe - Fast, Helpful AI Chat. Website. <https://poe.com/> Accessed: 2026-02-28.
- [44] Xiao Pu, Tianxing He, and Xiaojun Wan. 2024. Style-Compress: An LLM-based prompt compression framework considering task-specific styles. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. 14533–14549.
- [45] Qwen Team. 2024. Qwen Documentation. <https://qwen.readthedocs.io/en/latest/>.
- [46] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. 2016. Squad: 100,000+ questions for machine comprehension of text. *arXiv preprint arXiv:1606.05250* (2016).
- [47] David Rau, Shuai Wang, Hervé Déjean, and Stéphane Clinchant. 2024. Context embeddings for efficient answer generation in rag. *arXiv preprint arXiv:2407.09252* (2024).
- [48] Reddit User. 2024. Prompt Compression – LLM-lingua. https://www.reddit.com/r/LocalLLM/comments/1guambk/prompt_compression_llmlingua/.
- [49] Shivam Shandilya, Menglin Xia, Supriyo Ghosh, Huiqiang Jiang, Jue Zhang, Qianhui Wu, and Victor Rühle. 2024. Taco-rl: Task aware prompt compression optimization with reinforcement learning. *arXiv preprint arXiv:2409.13035* (2024).
- [50] Jiawen Shi, Zenghui Yuan, Yinyao Liu, Yue Huang, Pan Zhou, Lichao Sun, and Neil Zhenqiang Gong. 2024. Optimization-based prompt injection attack to llm-as-a-judge. In *Proceedings of the 2024 on ACM SIGSAC Conference on Computer and Communications Security*. 660–674.
- [51] Yuling Shi, Yichun Qian, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. 2025. Longcodezip: Compress long context for code language models. *arXiv preprint arXiv:2510.00446* (2025).
- [52] Gemma Team, Thomas Mesnard, Cassidy Hardin, Robert Dadashi, Surya Bhupatiraju, Shreya Pathak, Laurent Sifre, Morgane Rivière, Mihir Sanjay Kale, Juliette Love, et al. 2024. Gemma: Open models based on gemini research and technology. *arXiv preprint arXiv:2403.08295* (2024).
- [53] Llama Team. 2025. Llama 3 Model Card. https://github.com/meta-llama/llama3/blob/main/MODEL_CARD.md.
- [54] David Wingate, Mohammad Shoeybi, and Taylor Sorensen. 2022. Prompt compression and contrastive conditioning for controllability and toxicity reduction in language models. *arXiv preprint arXiv:2210.03162* (2022).

- [55] x1xhlol. 2026. system-prompts-and-models-of-ai-tools. <https://github.com/x1xhlol/system-prompts-and-models-of-ai-tools>
- [56] Yuan-An Xiao, Pengfei Gao, Chao Peng, and Yingfei Xiong. 2025. Improving the efficiency of LLM agent systems through trajectory reduction. *arXiv preprint arXiv:2509.23586* (2025).
- [57] Yuchong Xie, Zesen Liu, Mingyu Luo, Zhixiang Zhang, Kaikai Zhang, Zongjie Li, Ping Chen, Shuai Wang, and Dongdong She. 2025. QueryIPI: Query-agnostic Indirect Prompt Injection on Coding Agents. *arXiv preprint arXiv:2510.23675* (2025).
- [58] Yuchong Xie, Mingyu Luo, Zesen Liu, Zhixiang Zhang, Kaikai Zhang, Yu Liu, Zongjie Li, Ping Chen, Shuai Wang, and Dongdong She. 2025. On the Security of Tool-Invocation Prompts for LLM-Based Agentic Systems: An Empirical Risk Assessment. *arXiv preprint arXiv:2509.05755* (2025).
- [59] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. 2025. Qwen3 technical report. *arXiv preprint arXiv:2505.09388* (2025).
- [60] Siboyi, Yule Liu, Zhen Sun, Tianshuo Cong, Xinlei He, Jiaxing Song, Ke Xu, and Qi Li. 2024. Jailbreak attacks and defenses against large language models: A survey. *arXiv preprint arXiv:2407.04295* (2024).
- [61] Ivan Zakazov, Berke Argin, Oussama Gabouj, Kamel Charaf, Alexander Sharipov, Alexi Semiz, Lorenzo Drudi, Nicolas Baldwin, and Robert West. 2025. Cmprrs: Abstractive Token-Level Question-Agnostic Prompt Compressor. *arXiv preprint arXiv:2511.12281* (2025).
- [62] Yiming Zhang, Nicholas Carlini, and Daphne Ippolito. 2023. Effective prompt extraction from language models. *arXiv preprint arXiv:2307.06865* (2023).
- [63] Zhaowei Zhang, Hongyu Zhang, Beijun Shen, and Xiaodong Gu. 2022. Diet code is healthy: Simplifying programs for pre-trained models of code. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 1073–1084.
- [64] Sicheng Zhu, Ruiyi Zhang, Bang An, Gang Wu, Joe Barrow, Zichao Wang, Furong Huang, Ani Nenkova, and Tong Sun. 2023. Autodan: interpretable gradient-based adversarial attacks on large language models. *arXiv preprint arXiv:2310.15140* (2023).
- [65] Shengyao Zhuang, Xueguang Ma, Bevan Koopman, Jimmy Lin, and Guido Zuccon. 2024. PromptReps: Prompting large language models to generate dense and sparse representations for zero-shot document retrieval. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. 4375–4391.
- [66] Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint arXiv:2307.15043* (2023).
- [67] Wei Zou, Runpeng Geng, Binghui Wang, and Jinyuan Jia. 2025. {PoisonedRAG}: Knowledge corruption attacks to {Retrieval-Augmented} generation of large language models. In *34th USENIX Security Symposium (USENIX Security 25)*. 3827–3844.